

1. Three use-cases when you need asynchronous programming

You use async programming when you want to avoid blocking threads while waiting on some long-running work. Some common scenarios:

1. I/O bound operations

- Example: calling a Web API, reading/writing files, database queries, network calls.
- You don't want your thread (especially UI thread / web request thread) to wait idly while I/O completes.

2. High-concurrency / scalable servers

- In web applications (ASP.NET), using async helps free up threads to serve more requests while some are waiting on I/O.
- Reduces thread starvation, improves throughput.

3. Parallel / background processing with smoother UX

- Example: running data processing tasks, computation, or background tasks (logging, caching, image processing) without making user wait.
- Also useful when triggering many independent tasks (e.g. downloading multiple files in parallel) to improve performance.

Because .NET's async/await + Task model is well supported, it becomes natural to use async in these cases.

2. What's new in .NET 9 regarding operations on generic types

.NET 9 introduces several enhancements relevant to generics and related operations:

- UnsafeAccessorAttribute now supports generic parameters
You can now use UnsafeAccessorAttribute to access private members in generic types (earlier only non-generic types). blog.vineforce.net
- New collection types / generic operations
 - ReadOnlySet<T> was introduced (immutable read-only set) as a complement to existing generic read-only types like ReadOnlyDictionary etc. [PVS-Studio](https://pvs-studio.com)
 - In LINQ, new methods like CountBy and AggregateBy allow you to group and aggregate without needing to first make expensive groupings. [Microsoft Learn](https://learn.microsoft.com)
- Support for ref struct generic parameters
C# 13 (landing in .NET 9 era) adds support so that ref struct can be used as a generic parameter in more places. [PVS-Studio](https://pvs-studio.com)

These features make generic code more powerful, flexible, and efficient.

3s. Delegate built-in types in detail

In C#, delegates are types that represent references to methods (method pointers) with specific signatures. The framework provides several built-in delegate types. The most common ones are:

1. Action

- A delegate type that returns void.
- Can have 0 to many generic parameters for input arguments.
 - Action()
 - Action<T1>
 - Action<T1, T2>
 - ... up to Action<T1, ..., T16> (in .NET)
- Example: Action<int, string> means a method taking (int, string) and returning void.

2. Func

- A delegate type that returns a value (of generic type) and can take parameters.
- The last generic parameter is the return type.
 - Func<TResult> (no input, returns TResult)
 - Func<T1, TResult>
 - Func<T1, T2, TResult>
 - ... up to Func<T1, ..., T16, TResult>
- Example: Func<int, string, bool> means a method taking (int, string) and returning bool.

3. Predicate<T>

- A specialized Func<T, bool> — takes one input T and returns bool.
- Typically used for conditions or filters.
 - Example: Predicate<Student> means a method bool IsEligible(Student s).

4. Comparison<T>

- A delegate delegate int Comparison<T>(T x, T y)
- Used in sorting, comparing two objects of type T. Returns negative / zero / positive depending on ordering.